

flagquiz

Day 1 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“find the answer that is easiest to change into any other answer”*
- ▶ *“smallest maximum amount of changes”*

Programming interpretation:

- ▶ Treat each answer as an ordered list of parts split by comma-space.
- ▶ The distance between two answers is the number of positions where their parts differ.
- ▶ For each answer, compute its worst distance to all other answers, then print the minimum-worst candidates.

Textbook connection: CP4 2.2 d. Array Manipulation, Harder

Core technique: 2D arrays and pairwise comparison

- ▶ Each possible answer is a row in a 2D array of strings.
- ▶ The incongruity of one answer is its maximum difference against any other answer.
- ▶ The output is every answer with the smallest possible incongruity, preserving input order.

- ▶ N is small enough for all-pairs comparison.
- ▶ Comparing two answer rows is just a coordinate-by-coordinate mismatch count.
- ▶ The problem asks for a minimax value: minimize the worst distance to another answer.

- ▶ Read all answer lines and split each line by comma-space.
- ▶ For each candidate answer i , compare it with every answer j .
- ▶ Track the largest mismatch count seen for candidate i .
- ▶ Maintain the global smallest largest-mismatch value.
- ▶ Print candidates whose largest mismatch equals the global best.

- ▶ `countDifferences` uses `zip` so corresponding fields are compared directly.
- ▶ If two rows are identical, their distance is zero.
- ▶ When a candidate beats the current best, reset the output list.
- ▶ When a candidate ties the current best, append it to the output list.

- ▶ Every possible answer is considered as the candidate final answer.
- ▶ For each candidate, every possible target answer is checked, so the computed maximum is exact.
- ▶ Choosing the candidates with minimum exact maximum difference matches the required least incongruous alternatives.

Complexity: $O(N^2 * M)$ time, where M is the number of fields per answer; $O(N * M)$ memory.

- ▶ Keep rows as lists while comparing, then join only when preparing output.
- ▶ Do not sort the candidates; ties must stay in input order.
- ▶ The solution is intentionally direct because the constraints make it safe.

Source lines: flagquiz.py:27-35

```
import sys

def countDifferences(ans1, ans2):
    count = 0
    for op1, op2 in zip(ans1, ans2):
        if op1 != op2:
            count += 1
    return count
```

What this chunk does

- ▶ The helper performs a position-by-position comparison.
- ▶ zip pairs corresponding fields, matching the statement rule that order matters.
- ▶ The returned count is the number of edits between two alternatives.

Source lines: flagquiz.py:38-45

```
data = sys.stdin.read().split("\n")

N = int(data[1])
output = []
answers = []

for line in data[2 : N + 2]:
    answers.append(line.split(", "))
```

What this chunk does

- ▶ The question line is ignored because only the alternatives affect the answer.
- ▶ Each answer line becomes a list of parts, so later comparisons are direct list comparisons.
- ▶ The input guarantees all alternatives have the same number of parts.

Source lines: flagquiz.py:47-56

```
lowest = float("inf")
for i1 in range(len(answers)):
    instanceGreatest = float("-inf")
    for i2 in range(len(answers)):
        if i1 != i2 and answers[i1] == answers[i2]:
            instanceGreatest = 0
        else:
            diffTest = countDifferences(answers[i1], answers[i2])
            if diffTest > instanceGreatest:
                instanceGreatest = diffTest
```

What this chunk does

- ▶ The outer loop selects one answer as the candidate.
- ▶ The inner loop compares it against every possible answer.
- ▶ instanceGreatest records the candidate's incongruousity: its worst observed difference.

Source lines: flagquiz.py:57-63

```
if instanceGreatest < lowest:
    lowest = instanceGreatest
    output = [", ".join(list(map(str, answers[i1])))])
elif instanceGreatest == lowest:
    output.append(", ".join(list(map(str, answers[i1])))])

sys.stdout.write("\n".join(output))
```

What this chunk does

- ▶ A strictly better candidate resets the answer list.
- ▶ A tie is appended, preserving input order.
- ▶ The stored list form is joined back into the required comma-separated output.

Solution file: `practice_contests/day_1/flagquiz.py`

Problem: <https://open.kattis.com/problems/flagquiz>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.